claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a68b0271182268f2e.jsonl`
- SHA-256: `0b93325d247e401b16b2fa8aac2b0415d2850a24acd314fb0937e5be04f4f8a1`
- Source modified: `2026-06-10T06:13:52+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T06:10:54.966Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("reliability") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"Concurrent runs of the same video collide on shared, hardcoded working paths and delete each other's in-flight files","severity":"high","area":"concurrency / resource lifecycle","file":"backend/pipeline/segmenters/gemini.py:70/141 (temp_trimmed_{video_id}.mp4, chunk_{video_id}_{ci}.mp4), yolo_hybrid.py:356/490, tracknet_hybrid.py:184, wasb_hybrid.py:164 (handoff_{video_id}_{wi}.mp4), wasb_runner.py:87/122 (~/clips/<base>, <stem>_frames)","evidence":"All intermediate artifact names are derived only from video_id (+chunk/window index) under a fixed output/ tree; the WSL stage path is ~/clips/<basename>. Two concurrent processings of the same file write to identical chunk_*/handoff_* paths, and each run does os.remove(w_path) (e.g. wasb_hybrid.py:180) and os.rmdir(handoff_dir) (wasb_hybrid.py:192) ? so one run deletes the other's clip mid-flight or removes its directory.","why_it_matters":"Same-video double-submit (a trivially common retry/idempotency case) produces corrupted or empty highlight reels nondeterministically. The async-job plan (DEFERRED #16) discusses idempotency keys but not the on-disk filename collisions, so this race survives even once jobs are queued if two distinct jobs share a video_id.","why":"placeholder"}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:11:03.308Z)

```
backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
```

3. user (2026-06-10T06:11:04.586Z)

```
backend\config\models.py:52:    chunk_duration: float = 90.0
backend\config\models.py:53:    chunk_overlap: float = 10.0
backend\config\models.py:60:    yolo_chunk_workers: int = 2
backend\config\models.py:64:    ai_handoff_padding: float = 2.0
backend\infrastructure\database.py:79:                          chunk_index INTEGER,
backend\infrastructure\database.py:144:                                 chunk_index: Optional[int]
= None, confidence: Optional[float] = None,
backend\infrastructure\database.py:154:                         (video_id, detector, chunk_index,
start_time, end_time, duration,
backend\infrastructure\database.py:157:                     (video_id, detector, chunk_index,
start_time, end_time, end_time - start_time,
backend\sports\badminton.py:8:    def get_prompt(self, chunk_duration: float = 0.0) -> str:
backend\sports\badminton.py:56:        if chunk_duration > 0.0:
backend\sports\badminton.py:58:                f"IMPORTANT VIDEO CONTEXT: This video segment is
exactly {chunk_duration:.1f} seconds long. "
backend\sports\badminton.py:59:                f"All timestamps MUST be between 0 and
{chunk_duration:.1f}. "
backend\sports\badminton.py:60:                f"Any timestamp beyond {chunk_duration:.1f}s is
INVALID.\n\n"
backend\sports\base.py:11:    def get_prompt(self, chunk_duration: float = 0.0) -> str:
backend\providers\base.py:11:                       chunk_duration: float = 0.0,
backend\providers\base.py:19:            chunk_duration: Duration of this chunk in seconds, or
0.0 if not chunked.
backend\eval\gemini_refine.py:86:        prompt = sport_profile.get_prompt(chunk_duration=ce -
cs)
backend\eval\gemini_refine.py:87:        raw, _ = provider.analyze_video(clip, prompt,
chunk_duration=ce - cs, label=f"cand{idx}")
backend\eval\gemini_refine.py:151:                      min_dur: float = 2.0, chunk_sec: float
= 120.0, overlap: float = 10.0,
backend\eval\gemini_refine.py:164:        t += max(1.0, chunk_sec - overlap)
backend\eval\gemini_refine.py:169:        cend = min(cstart + chunk_sec, duration)
backend\eval\gemini_refine.py:170:        clip = osp.join(tmpdir, f"chunk_{ci:03d}.mp4")
backend\eval\gemini_refine.py:175:            prompt =
sport_profile.get_prompt(chunk_duration=cend - cstart)
backend\eval\gemini_refine.py:176:            raw, _ = provider.analyze_video(clip, prompt,
chunk_duration=cend - cstart, label=f"chunk{ci}")
backend\eval\gemini_refine.py:184:    print(f"[gemini_full] {len(starts)} chunk(s) of {video}
(<= {chunk_sec}s, width {clip_scale_width}) "
backend\providers\gemini.py:29:                       chunk_duration: float = 0.0,
backend\pipeline\segmenters\gemini.py:64:        temp_trimmed_path = None
backend\pipeline\segmenters\gemini.py:70:            temp_trimmed_path = os.path.join("output",
f"temp_trimmed_{video_id}.mp4")
backend\pipeline\segmenters\gemini.py:72:            if os.path.exists(temp_trimmed_path):
backend\pipeline\segmenters\gemini.py:74:                    os.remove(temp_trimmed_path)
backend\pipeline\segmenters\gemini.py:84:                temp_trimmed_path
backend\pipeline\segmenters\gemini.py:89:                logger.info(f"Trimmed video
successfully created at: {temp_trimmed_path}")
backend\pipeline\segmenters\gemini.py:90:                video_to_upload = temp_trimmed_path
backend\pipeline\segmenters\gemini.py:99:            if is_trimmed and temp_trimmed_path and
os.path.exists(temp_trimmed_path):
backend\pipeline\segmenters\gemini.py:101:                   os.remove(temp_trimmed_path)
backend\pipeline\segmenters\gemini.py:114:        chunk_duration = idx_cfg.get("chunk_duration",
300)    # 5 minutes default
backend\pipeline\segmenters\gemini.py:115:        chunk_overlap = idx_cfg.get("chunk_overlap",
30)       # 30 seconds overlap
```

```
backend\pipeline\segmenters\gemini.py:116:        use_chunking = video_duration > 0 and
video_duration > chunk_duration
backend\pipeline\segmenters\gemini.py:123:            step = chunk_duration - chunk_overlap
backend\pipeline\segmenters\gemini.py:124:            chunk_starts = []
backend\pipeline\segmenters\gemini.py:127:                chunk_starts.append(t)
backend\pipeline\segmenters\gemini.py:129:            num_chunks = len(chunk_starts)
backend\pipeline\segmenters\gemini.py:132:                f"({chunk_duration}s each,
{chunk_overlap}s overlap)."
backend\pipeline\segmenters\gemini.py:137:            def process_single_chunk(ci: int,
chunk_start: float) -> Tuple[str, list, dict]:
backend\pipeline\segmenters\gemini.py:138:                chunk_end = min(chunk_start +
chunk_duration, video_duration)
backend\pipeline\segmenters\gemini.py:139:                actual_chunk_dur = chunk_end -
chunk_start
backend\pipeline\segmenters\gemini.py:140:                chunk_label = f"Chunk
{ci+1}/{num_chunks}"
backend\pipeline\segmenters\gemini.py:141:                chunk_path = os.path.join(chunks_dir,
f"chunk_{video_id}_{ci}.mp4")
backend\pipeline\segmenters\gemini.py:144:                logger.info(f"Extracting
{chunk_label}: {chunk_start:.1f}s - {chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
backend\pipeline\segmenters\gemini.py:145:                success =
extract_video_chunk(video_to_upload, chunk_start, actual_chunk_dur, chunk_path)
backend\pipeline\segmenters\gemini.py:147:                    logger.error(f"Failed to extract
{chunk_label}. Skipping.")
backend\pipeline\segmenters\gemini.py:148:                    return chunk_label, [], {}
backend\pipeline\segmenters\gemini.py:152:                    chunk_size_mb =
os.path.getsize(chunk_path) / (1024 * 1024)
backend\pipeline\segmenters\gemini.py:154:                    chunk_size_mb = 0.0
backend\pipeline\segmenters\gemini.py:157:                prompt =
sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
backend\pipeline\segmenters\gemini.py:160:                chunk_segments, metrics =
provider.analyze_video(
backend\pipeline\segmenters\gemini.py:161:                    chunk_path, prompt,
chunk_duration=actual_chunk_dur, label=chunk_label
backend\pipeline\segmenters\gemini.py:163:                metrics["file_size_mb"] =
chunk_size_mb
backend\pipeline\segmenters\gemini.py:164:                metrics["segments_found"] =
len(chunk_segments)
backend\pipeline\segmenters\gemini.py:167:                for segment in chunk_segments:
backend\pipeline\segmenters\gemini.py:170:                            segment["start_time"] =
float(segment["start_time"]) + chunk_start
backend\pipeline\segmenters\gemini.py:175:                            segment["end_time"] =
float(segment["end_time"]) + chunk_start
backend\pipeline\segmenters\gemini.py:179:                logger.info(f"{chunk_label} returned
{len(chunk_segments)} play segments.")
backend\pipeline\segmenters\gemini.py:183:                    os.remove(chunk_path)
backend\pipeline\segmenters\gemini.py:187:                return chunk_label, chunk_segments,
metrics
backend\pipeline\segmenters\gemini.py:194:                futures =
[executor.submit(process_single_chunk, ci, start) for ci, start in enumerate(chunk_starts)]
backend\pipeline\segmenters\gemini.py:243:            prompt =
sport_profile.get_prompt(chunk_duration=video_duration)
backend\pipeline\segmenters\gemini.py:245:                video_to_upload, prompt,
chunk_duration=video_duration, label="SingleFile"
backend\pipeline\detectors\tracknet_runner.py:236:        chunk_start: float = 0.0,
backend\pipeline\detectors\tracknet_runner.py:240:        chunk_index: Optional[int] = None,
backend\pipeline\detectors\tracknet_runner.py:280:                "start": group[0][0] / fps +
chunk_start,
backend\pipeline\detectors\tracknet_runner.py:281:                "end": group[-1][0] / fps +
chunk_start,
backend\pipeline\detectors\tracknet_runner.py:286:                "chunk_index": chunk_index,
backend\pipeline\segmenters\tracknet_hybrid.py:102:        handoff_padding =
idx_cfg.get("ai_handoff_padding", 2.0)
backend\pipeline\segmenters\tracknet_hybrid.py:138:            points, fps=fps,
frame_width=frame_width, chunk_start=0.0,
```

```
backend\pipeline\segmenters\tracknet_hybrid.py:140:
min_window_duration=min_window_duration, chunk_index=0,
backend\pipeline\segmenters\tracknet_hybrid.py:167:
chunk_index=w["chunk_index"], confidence=confidence,
backend\pipeline\segmenters\tracknet_hybrid.py:175:        handoff_dir = os.path.join("output",
f"chunks_{provider_name}_handoff")
backend\pipeline\segmenters\tracknet_hybrid.py:176:        os.makedirs(handoff_dir,
exist_ok=True)
backend\pipeline\segmenters\tracknet_hybrid.py:181:            w_start = max(0, window["start"]
- handoff_padding)
backend\pipeline\segmenters\tracknet_hybrid.py:182:            w_end = min(video_duration,
window["end"] + handoff_padding)
backend\pipeline\segmenters\tracknet_hybrid.py:184:            w_path =
os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
backend\pipeline\segmenters\tracknet_hybrid.py:187:            prompt =
sport_profile.get_prompt(chunk_duration=w_dur)
backend\pipeline\segmenters\tracknet_hybrid.py:188:            segments, _ =
provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
backend\pipeline\segmenters\tracknet_hybrid.py:213:            os.rmdir(handoff_dir)
backend\pipeline\segmenters\wasb_hybrid.py:93:        handoff_padding =
idx_cfg.get("ai_handoff_padding", 2.0)
backend\pipeline\segmenters\wasb_hybrid.py:125:            points, fps=fps,
frame_width=frame_width, chunk_start=0.0,
backend\pipeline\segmenters\wasb_hybrid.py:127:
min_window_duration=min_window_duration, chunk_index=0,
backend\pipeline\segmenters\wasb_hybrid.py:147:
chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
backend\pipeline\segmenters\wasb_hybrid.py:155:        handoff_dir = os.path.join("output",
f"chunks_{provider_name}_handoff")
backend\pipeline\segmenters\wasb_hybrid.py:156:        os.makedirs(handoff_dir, exist_ok=True)
backend\pipeline\segmenters\wasb_hybrid.py:161:            w_start = max(0, window["start"] -
handoff_padding)
backend\pipeline\segmenters\wasb_hybrid.py:162:            w_end = min(video_duration,
window["end"] + handoff_padding)
backend\pipeline\segmenters\wasb_hybrid.py:164:            w_path = os.path.join(handoff_dir,
f"handoff_{video_id}_{wi}.mp4")
backend\pipeline\segmenters\wasb_hybrid.py:167:            prompt =
sport_profile.get_prompt(chunk_duration=w_dur)
backend\pipeline\segmenters\wasb_hybrid.py:168:            segments, _ =
provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
backend\pipeline\segmenters\wasb_hybrid.py:192:            os.rmdir(handoff_dir)
backend\pipeline\segmenters\yolo_hybrid.py:142:    def _extract_action_windows_from_chunk(self,
chunk_path: str, chunk_start: float,
backend\pipeline\segmenters\yolo_hybrid.py:146:
chunk_index: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\yolo_hybrid.py:151:            track_id_count, chunk_index.
backend\pipeline\segmenters\yolo_hybrid.py:158:            chunk_index: Index of the source
chunk (recorded on each window for traceability).
backend\pipeline\segmenters\yolo_hybrid.py:160:        cap = cv2.VideoCapture(chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:162:            logger.error(f"Could not open
{chunk_path}")
backend\pipeline\segmenters\yolo_hybrid.py:238:                "start": group[0][0] / fps +
chunk_start,
backend\pipeline\segmenters\yolo_hybrid.py:239:                "end": group[-1][0] / fps +
chunk_start,
backend\pipeline\segmenters\yolo_hybrid.py:244:                "chunk_index": chunk_index,
backend\pipeline\segmenters\yolo_hybrid.py:303:        chunk_duration =
idx_cfg.get("chunk_duration", 300)
backend\pipeline\segmenters\yolo_hybrid.py:304:        chunk_overlap =
idx_cfg.get("chunk_overlap", 30)
backend\pipeline\segmenters\yolo_hybrid.py:309:        handoff_padding =
idx_cfg.get("ai_handoff_padding", idx_cfg.get("gemini_handoff_padding", 2.0))
backend\pipeline\segmenters\yolo_hybrid.py:329:        step = chunk_duration - chunk_overlap
backend\pipeline\segmenters\yolo_hybrid.py:330:        chunk_starts = []
backend\pipeline\segmenters\yolo_hybrid.py:333:            chunk_starts.append(t)
```

```
backend\pipeline\segmenters\yolo_hybrid.py:336:        num_chunks = len(chunk_starts)
backend\pipeline\segmenters\yolo_hybrid.py:354:                chunk_end = min(cs +
chunk_duration, video_duration)
backend\pipeline\segmenters\yolo_hybrid.py:355:                actual_dur = chunk_end - cs
backend\pipeline\segmenters\yolo_hybrid.py:356:                chunk_path =
os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
backend\pipeline\segmenters\yolo_hybrid.py:357:                success =
extract_video_chunk(video_path, cs, actual_dur, chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:358:                return (ci, cs, chunk_path,
success)
backend\pipeline\segmenters\yolo_hybrid.py:363:                for ci, cs in
enumerate(chunk_starts)
backend\pipeline\segmenters\yolo_hybrid.py:368:                ci, chunk_start, chunk_path,
success = future.result()
backend\pipeline\segmenters\yolo_hybrid.py:369:                chunk_end = min(chunk_start +
chunk_duration, video_duration)
backend\pipeline\segmenters\yolo_hybrid.py:370:                logger.info(f"Processing Chunk
{ci+1}/{num_chunks} ({chunk_start:.1f}s - {chunk_end:.1f}s)...")
backend\pipeline\segmenters\yolo_hybrid.py:377:                    chunk_path, chunk_start,
velocity_thresh, merge_gap,
backend\pipeline\segmenters\yolo_hybrid.py:378:                    frame_skip=frame_skip,
tracker_config=tracker_config, chunk_index=ci
backend\pipeline\segmenters\yolo_hybrid.py:384:                    os.remove(chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:391:            for ci, chunk_start in
enumerate(chunk_starts):
backend\pipeline\segmenters\yolo_hybrid.py:392:                chunk_end = min(chunk_start +
chunk_duration, video_duration)
backend\pipeline\segmenters\yolo_hybrid.py:393:                actual_dur = chunk_end -
chunk_start
backend\pipeline\segmenters\yolo_hybrid.py:394:                chunk_path =
os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
backend\pipeline\segmenters\yolo_hybrid.py:396:                logger.info(f"Processing Chunk
{ci+1}/{num_chunks} ({chunk_start:.1f}s - {chunk_end:.1f}s)...")
backend\pipeline\segmenters\yolo_hybrid.py:397:                success =
extract_video_chunk(video_path, chunk_start, actual_dur, chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:402:                    chunk_path, chunk_start,
velocity_thresh, merge_gap,
backend\pipeline\segmenters\yolo_hybrid.py:403:                    frame_skip=frame_skip,
tracker_config=tracker_config, chunk_index=ci
backend\pipeline\segmenters\yolo_hybrid.py:409:                    os.remove(chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:414:        # chunk_overlap, so the same rally can
surface in two adjacent chunks).
backend\pipeline\segmenters\yolo_hybrid.py:426:                "chunk_index": a["chunk_index"],
backend\pipeline\segmenters\yolo_hybrid.py:469:                    chunk_index=w["chunk_index"],
confidence=confidence,
backend\pipeline\segmenters\yolo_hybrid.py:486:            w_start = max(0, window["start"] -
handoff_padding)
backend\pipeline\segmenters\yolo_hybrid.py:487:            w_end = min(video_duration,
window["end"] + handoff_padding)
backend\pipeline\segmenters\yolo_hybrid.py:490:            w_path = os.path.join(gemini_dir,
f"handoff_{video_id}_{wi}.mp4")
backend\pipeline\segmenters\yolo_hybrid.py:496:            prompt =
sport_profile.get_prompt(chunk_duration=w_dur)
backend\pipeline\segmenters\yolo_hybrid.py:497:            segments, metrics =
provider.analyze_video(w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi+1}")
```

4. user (2026-06-10T06:11:14.395Z)

```
150
151             if not all_candidate_windows:
152                 return []
153
154             # --- Phase 3: AI provider handoff (shared pattern) ---
155             handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
156             os.makedirs(handoff_dir, exist_ok=True)
```

```
157             logger.info("Phase 3: %s validation on %d candidate windows...",
158                         provider_name, len(all_candidate_windows))
159
160             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
161                 w_start = max(0, window["start"] - handoff_padding)
162                 w_end = min(video_duration, window["end"] + handoff_padding)
163                 w_dur = w_end - w_start
164                 w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
165                 if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
166                     return []
167                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
168                 segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
169                                                      label=f"Handoff {wi+1}")
170                 valid = []
171                 for seg in segments:
172                     try:
173                         seg["start_time"] = float(seg["start_time"]) + w_start
174                         seg["end_time"] = float(seg["end_time"]) + w_start
175                         seg["_candidate_id"] = candidate_ids[wi]
176                         valid.append(seg)
177                     except (ValueError, TypeError, KeyError) as e:
178                         logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
179                 try:
180                     os.remove(w_path)
181                 except OSError:
182                     pass
183                 return valid
184
185             ai_segments: List[dict] = []
186             with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
187                 futures = [executor.submit(process_candidate_window, wi, cw)
188                            for wi, cw in enumerate(all_candidate_windows)]
189                 for future in futures:
190                     ai_segments.extend(future.result())
191             try:
192                 os.rmdir(handoff_dir)
193             except OSError:
194                 pass
195             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
196
197             # --- Phase 4: DB population (shared pattern) ---
198             segments_data = []
199             for segment in ai_segments:
200                 try:
201                     start = float(segment["start_time"])
202                     end = float(segment["end_time"])
203                     dur = end - start
204                     shot_count = segment.get("shuttle_exchange_count")
205                     try:
206                         shot_count = int(shot_count)
207                     except (ValueError, TypeError):
208                         shot_count = max(3, int(dur / 0.8))
209                     metadata = {"shot_count": shot_count,
```

5. user (2026-06-10T06:11:15.216Z)

```
backend\pipeline\detectors\rally_gate.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_runner.py
```

backend\pipeline\detectors__init__.py

6. user (2026-06-10T06:11:28.296Z)

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
12      import os
13      import shlex
14      import logging
15      import subprocess
16      from dataclasses import dataclass
17      from typing import Dict, Any, Optional, Tuple
18
19      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20      # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21      from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22      from backend.pipeline.detectors.base import DetectorRunner
23
24      logger = logging.getLogger(__name__)
25
26      # Windows path to the inference wrapper we stage into WSL.
27      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30      @dataclass
31      class WasbConfig:
32          """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/WASB-SBDT"
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "wasb"
37          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38          sport: str = "badminton"
39          wsl_stage_dir: str = "~/clips"
40          timeout_sec: int = 0  # 0 = no timeout
41
42          @classmethod
43          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
44              w = (idx_cfg or {}).get("wasb", {}) or {}
45              return cls(
46                  distro=w.get("wsl_distro", "Ubuntu"),
47                  repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
48                  conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
49                  conda_env=w.get("conda_env", "wasb"),
50                  weights_path=w.get("weights_path",
51                                     "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
52                  sport=w.get("sport", "badminton"),
53                  wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
54                  timeout_sec=int(w.get("timeout_sec", 0)),
55              )
56
57
```

```
    class WasbRunner(DetectorRunner):  # DetectorRunner ABC (finish item 6 / new item 1)
        def __init__(self, cfg: WasbConfig):
            self.cfg = cfg

        def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
            full = f"source {self.cfg.conda_sh} && {inner_cmd}"
            argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
            logger.debug("WSL exec: %s", full)
            return subprocess.run(argv, capture_output=True, text=True,
                                  timeout=(self.cfg.timeout_sec or None))

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
            cmd = (f"conda activate {self.cfg.conda_env} && "
                   f"python -c \"import torch; print('torch', torch.__version__, "
                   f"'cuda', torch.cuda.is_available())\"")
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? Windows + WSL2 required."
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
            return True, out

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            src_mnt = to_wsl_mnt_path(video_win_path)
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        def _stage_infer_script(self) -> bool:
            """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
            src_mnt = to_wsl_mnt_path(_INFER_WIN)
            cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
            res = self._wsl_bash(cmd)
            return res.returncode == 0 and "OK" in (res.stdout or "")

        def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
            """Run WASB on a video. Returns the Windows path to the trajectory CSV, or
None."""
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize for cross-platform basename (tests use Windows paths on Linux).
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem = os.path.splitext(base)[0]
            wsl_out_dir = to_wsl_mnt_path(output_win_dir)
            expected_csv_win = os.path.join(output_win_dir, f"{stem}_wasb.csv")

            if not self._stage_infer_script():
                logger.error("Could not stage wasb_infer.py into WSL.")
                return None
```

```
119             staged_video = self._stage_video(video_win_path, base)
120             if staged_video is None:
121                 return None
122             frames_dir = f"{self.cfg.wsl_stage_dir}/{stem}_frames"
123 
124             cmd = (
125                 f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
126                 f"python wasb_infer.py "
127                 f"--video {shlex.quote(staged_video)} "
128                 f"--frames_out_dir {shlex.quote(frames_dir)} "
129                 f"--weights {shlex.quote(self.cfg.weights_path)} "
130                 f"--sport {shlex.quote(self.cfg.sport)} "
131                 f"--out {shlex.quote(wsl_out_dir + '/' + stem + '_wasb.csv')}"
132             )
133             logger.info("Running WASB inference on %s ...", base)
134             try:
135                 res = self._wsl_bash(cmd)
136             except subprocess.TimeoutExpired:
137                 logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
138                 return None
139             if res.returncode != 0:
140                 logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
141                 return None
142             if not os.path.exists(expected_csv_win):
143                 logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
144                 return None
145             logger.info("WASB trajectory CSV: %s", expected_csv_win)
146             return expected_csv_win
147 
148         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
149         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
150         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
151 
```

7. user (2026-06-10T06:11:29.700Z)

```
288:    parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-
sampled frames for debugging")
352:    # CLI processing mode (no web UI needed)
```

8. assistant (2026-06-10T06:11:37.756Z)

wasb_runner.py:87 and :122 verified. Now the key counter-evidence: how processing is triggered (is concurrent same-video processing even possible?) and whether this is a documented deferral.

9. user (2026-06-10T06:11:39.404Z)

No matches found

10. user (2026-06-10T06:11:40.859Z)

No matches found

11. user (2026-06-10T06:11:47.651Z)

```
backend\main.py:49:@app.get("/", response_class=HTMLResponse)
backend\main.py:87:@app.get("/api/config")
backend\main.py:91:@app.post("/api/process")
backend\main.py:92:def process_video(req: ProcessRequest):
backend\main.py:172:@app.post("/api/query")
```

```
backend\main.py:183:@app.post("/api/compile")
backend\providers\base.py:10:    def analyze_video(self, video_path: str, prompt: str,
backend\providers\gemini.py:28:    def analyze_video(self, video_path: str, prompt: str,
backend\pipeline\segmenters\base.py:24:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
backend\pipeline\segmenters\motion.py:22:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\tracknet_hybrid.py:59:    def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None,
backend\pipeline\segmenters\tracknet_hybrid.py:180:        def process_candidate_window(wi: int,
window: Dict[str, Any]) -> List[dict]:
backend\pipeline\segmenters\wasb_hybrid.py:56:    def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None,
backend\pipeline\segmenters\wasb_hybrid.py:160:        def process_candidate_window(wi: int,
window: Dict[str, Any]) -> List[dict]:
backend\pipeline\segmenters\yolo_hybrid.py:263:    def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\yolo_hybrid.py:485:        def process_candidate_window(wi: int,
window: Dict[str, Any]) -> List[dict]:
backend\pipeline\segmenters\gemini.py:25:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\gemini.py:137:            def process_single_chunk(ci: int,
chunk_start: float) -> Tuple[str, list, dict]:
```

12. user (2026-06-10T06:11:49.237Z)

```
docs\BACKLOG.md:67:- ? **Sequencing (Config first, then atomic refactor)** ? Implement
Deliverable 1 (Config) **first**, introducing the `backend/config/` package immediately. Perform
the full directory structure refactor as a **separate high-pri atomic PR** afterwards. This
minimizes conflicts, keeps PRs focused, and gives Claude a clean window with no concurrent
moves.
docs\DEFERRED_HARDENING_PLAN.md:90:## 3. Async job model (debt #16)
docs\DEFERRED_HARDENING_PLAN.md:101:  real queue (RQ/Celery) for later if multi-process is
needed.
docs\CODE_MAP.md:97:[Omitted long matching line]
docs\CODE_MAP.md:110:[Omitted long matching line]
docs\CODE_MAP.md:111:-
`@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
docs\CODE_MAP.md:167:[Omitted long matching line]
docs\CODE_MAP.md:193:`detection_params` keys by detector:
wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet
adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
docs\GOLDEN_SET_VENDORING.md:81:    def poll(self, job_id) -> str:
# queued|running|done|failed
docs\archives\COMMERCIAL_READINESS_REVIEW.md:90:   - Async job model, progress, storage,
auth/tenancy basics.
docs\NEXT_STEPS.md:46:- ? No owned-model implementation until owner greenlights a milestone.
Committed next PLATFORM slice = async job model (single-tenant).
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:165:next platform slice remains the async job model
(single-tenant); only the M0.1?M0.4 foundation is safe to start.*
docs\PLATFORM_ARCHITECTURE.md:41:> first infra slice (an async job model) is the on-hold #16 in
docs\PLATFORM_ARCHITECTURE.md:218:[`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md)
#16** (async job model) ? build
docs\PLATFORM_ARCHITECTURE.md:219:that first, single-tenant, then generalize.
docs\PLATFORM_ARCHITECTURE.md:222:  status (queued|running|done|failed), progress,
compute_backend, result_ref, error,
docs\PLATFORM_ARCHITECTURE.md:224:- **Idempotency** keyed on `(tenant_id, video_id,
pipeline_version)` ? reuse the
docs\PLATFORM_ARCHITECTURE.md:226:- **Dispatch:** the orchestrator enqueues; a worker (hosted or
`byo_worker`) claims jobs
```

```
docs\PLATFORM_ARCHITECTURE.md:230:- **Queue/worker backend ? a decision to prototype in #16,
*not* a default here.** The
docs\PLATFORM_ARCHITECTURE.md:231:  *requirements* are firm ? **idempotency**, **resumability**
(a 40-min GPU job mustn't
docs\PLATFORM_ARCHITECTURE.md:236:    Postgres-backed queue + worker** ? likely sufficient for
the *first* single-tenant
docs\PLATFORM_ARCHITECTURE.md:239:    retries/timeouts/concurrency; pairs neatly with the pull-
based worker) ? attractive
docs\PLATFORM_ARCHITECTURE.md:398:   job (§5 / DEFERRED #16): one config, idempotent, logged.
docs\PLATFORM_ARCHITECTURE.md:462:| **P0** | **Async job model, single-tenant** (DEFERRED #16) |
`jobs` table via `user_version` ladder; worker; 202+status endpoints |
docs\PLATFORM_ARCHITECTURE.md:555:[Omitted long matching line]
docs\PLATFORM_ARCHITECTURE.md:567:- [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) ?
#16 async job model (the P0 infra slice), #7 packaging.
docs\README.md:10:- [DEFERRED_HARDENING_PLAN.md](DEFERRED_HARDENING_PLAN.md) ? Design (pending
approval) for the larger deferred items: packaging, train/eval guardrails, and the async job
model (#16, the first infra slice toward the platform).
docs\TRACKNET_WSL_SETUP.md:63:  "queue_length": 5,
docs\archives\checkpoints\2026-06-strategy-foundation\README.md:18:-
**DEFERRED_HARDENING_PLAN.md** ? packaging, train/eval guardrails, async job model (#16) ?
*pending approval*.
docs\archives\checkpoints\2026-06-strategy-foundation\README.md:30:- First implementation slice
= **async job model (DEFERRED #16)**, single-tenant ? **pending explicit go-ahead** (no
implementation started without approval).
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:40:9. (Stretch) First
slice of async job model.
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:47:- Gives Claude a clean
window with no concurrent file moves.
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:168:- Async job model
slice.
```

13. user (2026-06-10T06:11:56.777Z)

```
85
86      # --- API Endpoints ---
87      @app.get("/api/config")
88      def get_config():
89          return global_config
90
91      @app.post("/api/process")
92      def process_video(req: ProcessRequest):
93          if not os.path.exists(req.video_path):
94              raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
95
96          video_id = compute_video_id(req.video_path)
97          was_reingest = db_instance.get_video(video_id) is not None
98
99          # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
100         logger.info(f"Running validations for {req.video_path}...")
101         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
102         failures = [r for r in val_results if not r.passed]
103
104         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
105         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
106         seg_name = req.segmenter or default_seg
107         segmenter_actual = None
108         segmentation_ran = False
109         response: Optional[Dict[str, Any]] = None
110         try:
111             if failures:
112                 db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
```

```
val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.dict() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            print(f"[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                available = list(segmenter_registry.list_available().keys())
                raise HTTPException(
                    status_code=400,
                    detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
                )

            print(f"[API] Using segmenter: {seg_name}")
            segmenter_actual = seg_name
            segmenter = segmenter_cls(db_instance, global_config)
            result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
            segmentation_ran = True

            if isinstance(result, Failure):
                print(f"[API] Segmenter failed: {result.message}")
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.dict() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.dict() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths

```

```
172     @app.post("/api/query")
173     def query_play_segments(req: QueryRequest):
174         filters = {
```

14. user (2026-06-10T06:12:39.868Z)

```
55                  provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
56              except KeyError as e:
57                  logger.error(str(e))
58                  return []
59
60              # A: Check for debug_duration to trim the video first
61              debug_duration = self.config.get("indexing", {}).get("debug_duration")
62              video_to_upload = video_path
63              is_trimmed = False
64              temp_trimmed_path = None
65              is_compressed = False
66              temp_compressed_path = None
67
68              if debug_duration:
69                  logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
70                  temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
71                  os.makedirs("output", exist_ok=True)
72                  if os.path.exists(temp_trimmed_path):
73                      try:
74                          os.remove(temp_trimmed_path)
75                      except Exception:
76                          pass
77
78                  cmd = [
79                      "ffmpeg", "-y",
80                      "-ss", "0",
81                      "-t", str(debug_duration),
82                      "-i", video_path,
83                      "-c", "copy",
84                      temp_trimmed_path
85                  ]
86                  logger.debug(f"Running command: {' '.join(cmd)}")
87                  try:
88                      subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
89                      logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
90                      video_to_upload = temp_trimmed_path
91                      is_trimmed = True
92                  except Exception as e:
93                      logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
94
95
96
97              # Helper to clean up local temp files
98              def cleanup_temp_files():
99                  if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
100                     try:
101                         os.remove(temp_trimmed_path)
102                     except Exception:
103                         pass
104
105             # 1. Determine actual video duration
106             video_duration = get_video_duration(video_to_upload)
107             if video_duration > 0:
108                 logger.info(f"Detected video duration: {video_duration:.1f}s")
```

```
            else:
                logger.warning("Could not detect video duration. Prompt will not include
duration context.")

            # 2. Decide whether to use chunked processing
            idx_cfg = self.config.get("indexing", {})
            chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
            chunk_overlap = idx_cfg.get("chunk_overlap", 30)       # 30 seconds overlap
            use_chunking = video_duration > 0 and video_duration > chunk_duration

            ai_segments = []
            chunks_dir = os.path.join("output", "chunks")

            if use_chunking:
                # --- CHUNKED PROCESSING ---
                step = chunk_duration - chunk_overlap
                chunk_starts = []
                t = 0.0
                while t < video_duration:
                    chunk_starts.append(t)
                    t += step
                num_chunks = len(chunk_starts)
                logger.info(
                    f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
                    f"({chunk_duration}s each, {chunk_overlap}s overlap)."
                )

                os.makedirs(chunks_dir, exist_ok=True)

                def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    actual_chunk_dur = chunk_end - chunk_start
                    chunk_label = f"Chunk {ci+1}/{num_chunks}"
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")

                    # Extract chunk with ffmpeg
                    logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
                    success = extract_video_chunk(video_to_upload, chunk_start,
actual_chunk_dur, chunk_path)
                    if not success:
                        logger.error(f"Failed to extract {chunk_label}. Skipping.")
                        return chunk_label, [], {}

                    # Measure chunk size
                    try:
                        chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
                    except Exception:
                        chunk_size_mb = 0.0

                    # Get prompt from sport profile
                    prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)

                    # Process this chunk through the provider
                    chunk_segments, metrics = provider.analyze_video(
                        chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
                    )
                    metrics["file_size_mb"] = chunk_size_mb
                    metrics["segments_found"] = len(chunk_segments)

                    # Offset timestamps back to full-video coordinates
                    for segment in chunk_segments:
```

```
168                         if "start_time" in segment:
169                             try:
170                                 segment["start_time"] = float(segment["start_time"]) +
chunk_start
171                             except (ValueError, TypeError):
172                                 pass
173                         if "end_time" in segment:
174                             try:
175                                 segment["end_time"] = float(segment["end_time"]) +
chunk_start
176                             except (ValueError, TypeError):
177                                 pass
178 
179                     logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
180 
181                     # Clean up chunk file
182                     try:
183                         os.remove(chunk_path)
184                     except Exception:
185                         pass
186 
187                     return chunk_label, chunk_segments, metrics
188 
189                 # Run chunks in parallel
190                 all_metrics = {}
191                 logger.info("Starting parallel processing with up to 5 concurrent
workers...")
192                 t_chunking_start = time.time()
193                 with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
194                     futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
195                     for future in concurrent.futures.as_completed(futures):
196                         label, segments, metrics = future.result()
197                         ai_segments.extend(segments)
198                         if metrics:
199                             all_metrics[label] = metrics
```

15. user (2026-06-10T06:12:40.549Z)

```
85      - Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
86      - Integration: a deliberately-leaked manifest fails the training entrypoint.
87
88      ---
89
90      ## 3. Async job model (debt #16)
91
92      ### Problem
93      `POST /api/process` runs validate ? segment ? report **inline**, blocking the
94      request for the full duration of a long video. Unworkable as a hosted/product API.
95
96      ### Approach (minimal first slice ? single self-hosted box)
97      - `POST /api/process` returns **202 + `job_id`**; add `GET /api/jobs/{job_id}`
98        returning `{status, progress, result, reports}`.
99      - Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`
100       (segmenters already use threads internally for AI handoff, so this fits); leave a
101       real queue (RQ/Celery) for later if multi-process is needed.
102     - Persist job state in the existing SQLite DB via the migration ladder
103       (`backend/infrastructure/database.py::_init_db`, bump `PRAGMA user_version` 1 ? 2,
104       add a `jobs` table). The observability report becomes the job's artifact.
105     - Update the static UI to poll `GET /api/jobs/{id}` instead of blocking.
106
107     ### Risks
108     - **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
109       `_get_connection` already re-issues `PRAGMA foreign_keys=ON` per connection ?
```

```
110       confirm it's safe under the executor.
111     - Cancellation/timeout and crash recovery (a job in RUNNING after a restart).
112     - In-process store is not horizontally scalable ? acceptable for the current
113       single-box self-hosted target; call it out so it isn't mistaken for production-HA.
114
115     ### Verification
116     - Submit a job ? poll ? DONE with the same segments the sync path produced.
117     - Server restart mid-job leaves a recoverable/clearly-FAILED job, not a hang.
118
119     ---
120
121     ## Out of this doc
122     - **Detector/WSL abstraction (#6):** in flight (PR #49).
123     - **Unify hybrid segmenters (#7 in LOW_REGRET):** owner-deferred.
124     - **ByteTrack `supervision<0.30.0` migration + real-footage validation (#14/#15):**
```

16. user (2026-06-10T06:13:13.782Z)

```
backend\pipeline\audio\hit_detector.py:144:            os.remove(wav)
backend\pipeline\segmenters\gemini.py:74:                  os.remove(temp_trimmed_path)
backend\pipeline\segmenters\gemini.py:101:                  os.remove(temp_trimmed_path)
backend\pipeline\segmenters\gemini.py:183:                  os.remove(chunk_path)
backend\pipeline\segmenters\gemini.py:229:               os.rmdir(chunks_dir)
backend\pipeline\segmenters\tracknet_hybrid.py:200:                os.remove(w_path)
backend\pipeline\segmenters\tracknet_hybrid.py:213:             os.rmdir(handoff_dir)
backend\pipeline\detectors\wasb_infer.py:149:            os.remove(p)
backend\pipeline\segmenters\wasb_hybrid.py:180:                 os.remove(w_path)
backend\pipeline\segmenters\wasb_hybrid.py:192:               os.rmdir(handoff_dir)
backend\pipeline\segmenters\yolo_hybrid.py:384:                    os.remove(chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:409:                    os.remove(chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:474:               os.rmdir(chunks_dir)
backend\pipeline\segmenters\yolo_hybrid.py:512:                   os.remove(w_path)
backend\pipeline\segmenters\yolo_hybrid.py:525:               os.rmdir(gemini_dir)
```

17. user (2026-06-10T06:13:46.926Z)

```
Structured output provided successfully
```

18. assistant (2026-06-10T06:13:52.153Z)

```
Verification complete.
```